

Evidencias

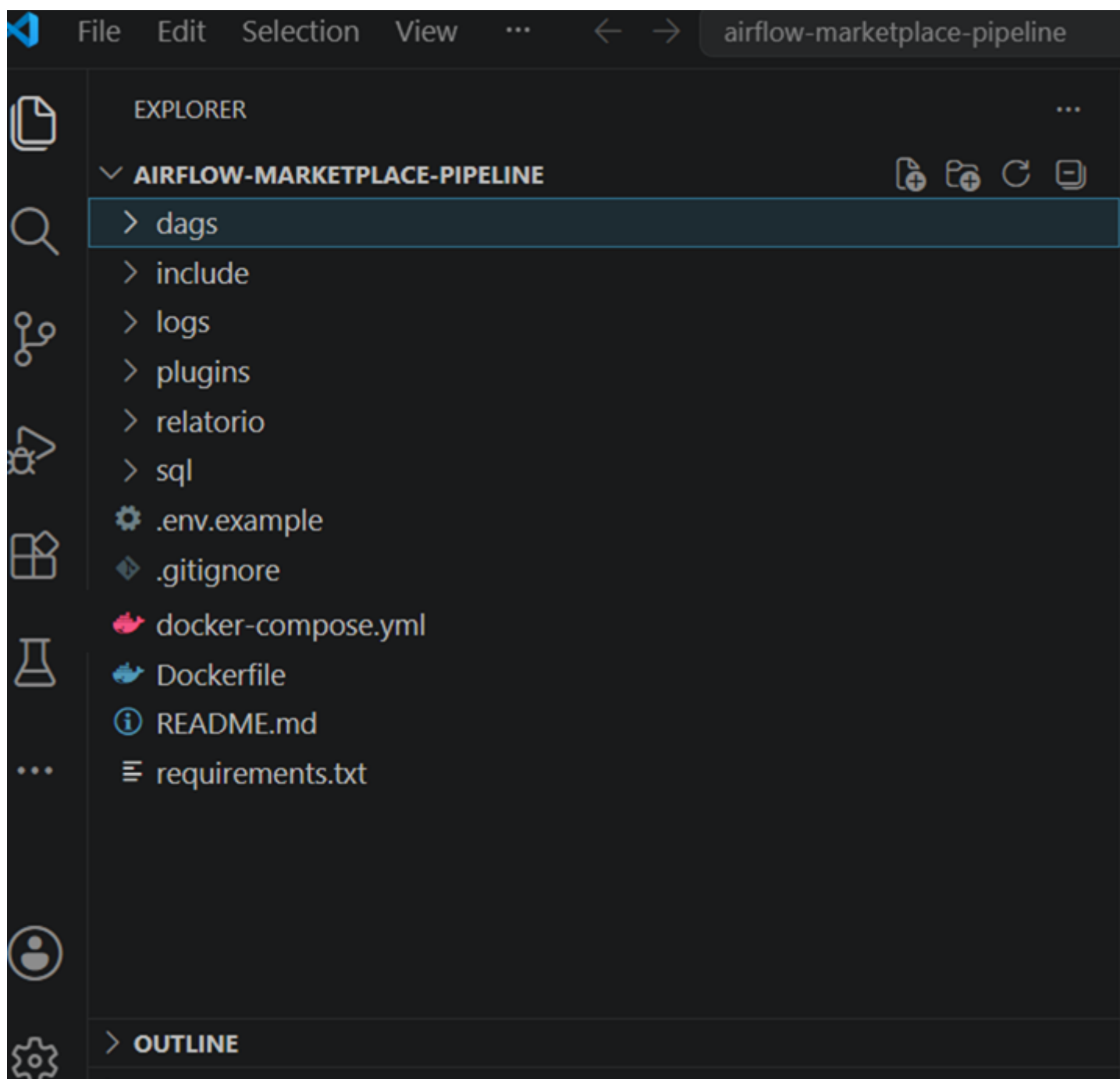
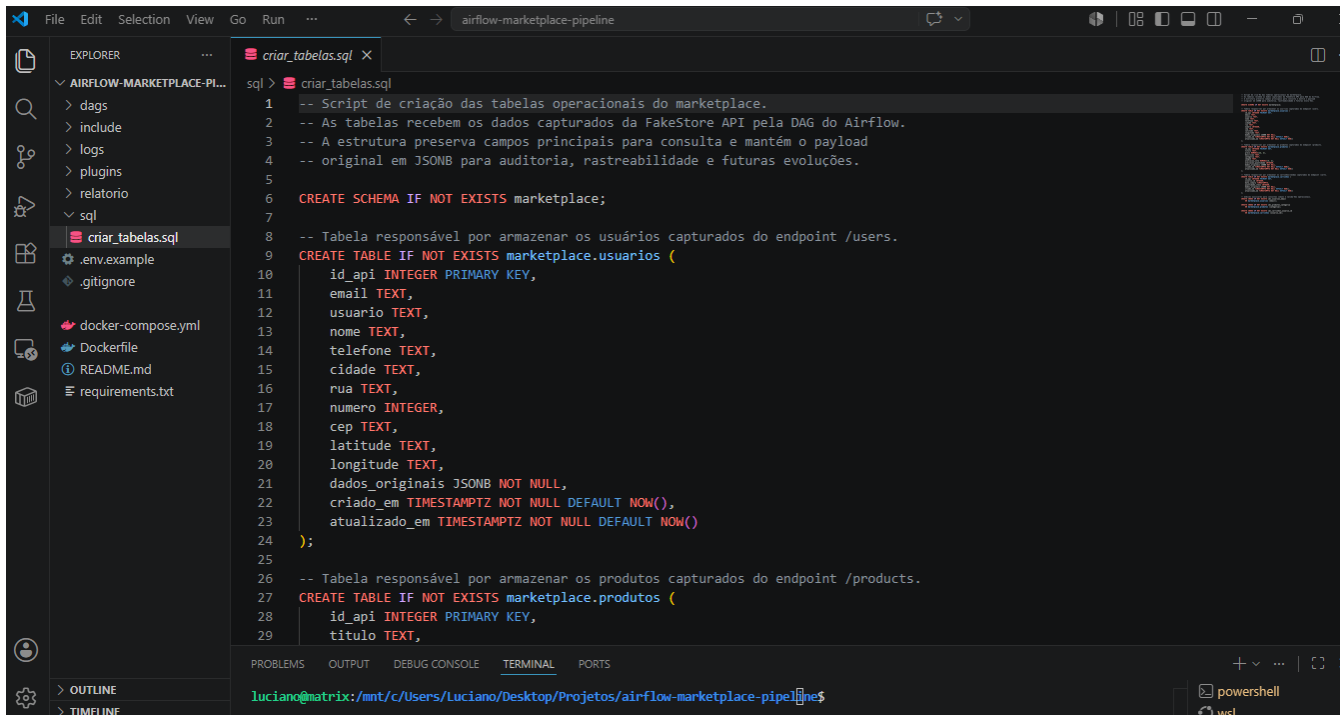


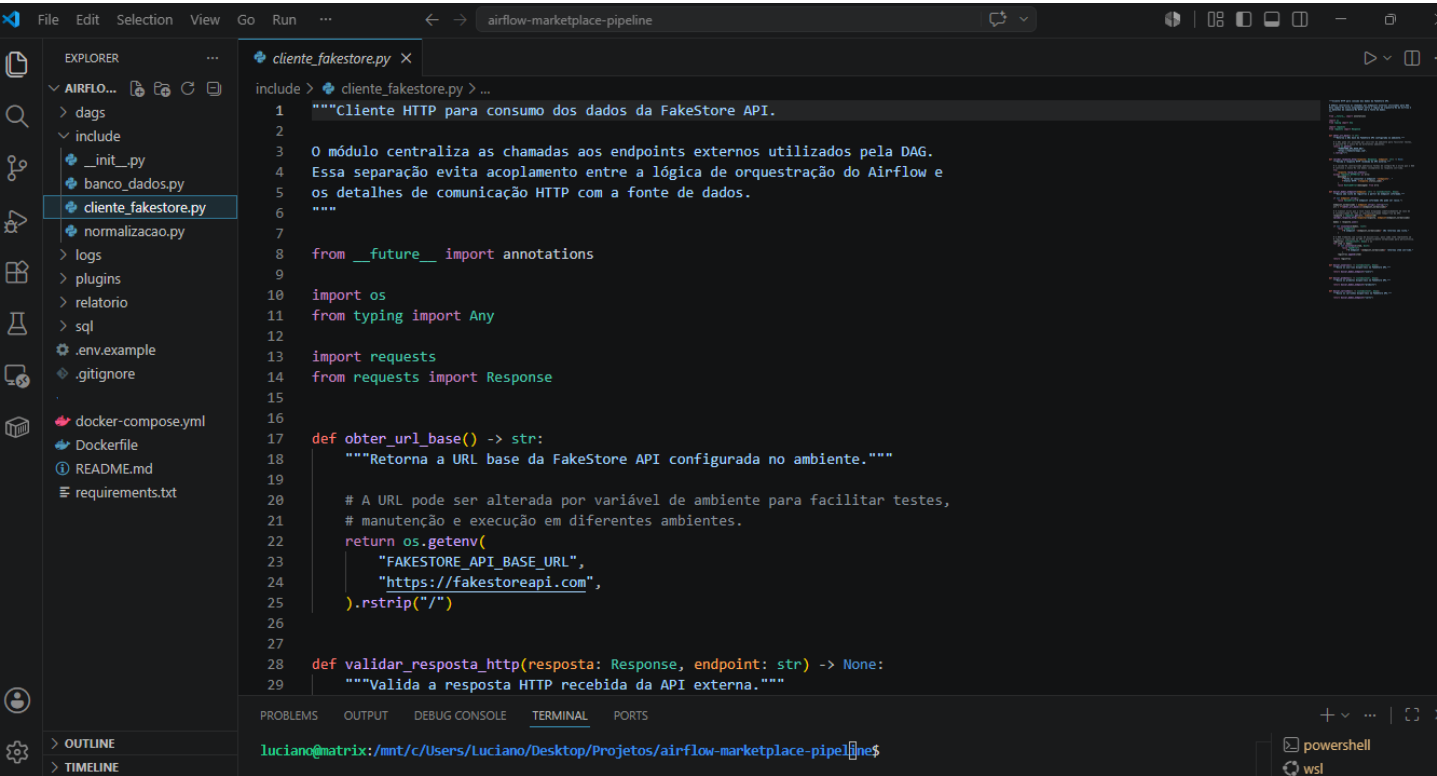
Figura 1 — Estrutura inicial do projeto airflow-marketplace-pipeline organizada no VS Code, com separação entre DAGs, módulos auxiliares, scripts SQL, configuração Docker, documentação e evidências.



```
1 -- Script de criação das tabelas operacionais do marketplace.
2 -- As tabelas recebem os dados capturados da FakeStore API pela DAG do Airflow.
3 -- A estrutura preserva campos principais para consulta e mantém o payload
4 -- original em JSONB para auditoria, rastreabilidade e futuras evoluções.
5
6 CREATE SCHEMA IF NOT EXISTS marketplace;
7
8 -- Tabela responsável por armazenar os usuários capturados do endpoint /users.
9 CREATE TABLE IF NOT EXISTS marketplace.usuarios (
10     id_api INTEGER PRIMARY KEY,
11     email TEXT,
12     usuario TEXT,
13     nome TEXT,
14     telefone TEXT,
15     cidade TEXT,
16     rua TEXT,
17     numero INTEGER,
18     cep TEXT,
19     latitude TEXT,
20     longitude TEXT,
21     dados_originais JSONB NOT NULL,
22     criado_em TIMESTAMPTZ NOT NULL DEFAULT NOW(),
23     atualizado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()
24 );
25
26 -- Tabela responsável por armazenar os produtos capturados do endpoint /products.
27 CREATE TABLE IF NOT EXISTS marketplace.produtos (
28     id_api INTEGER PRIMARY KEY,
29     titulo TEXT,
```

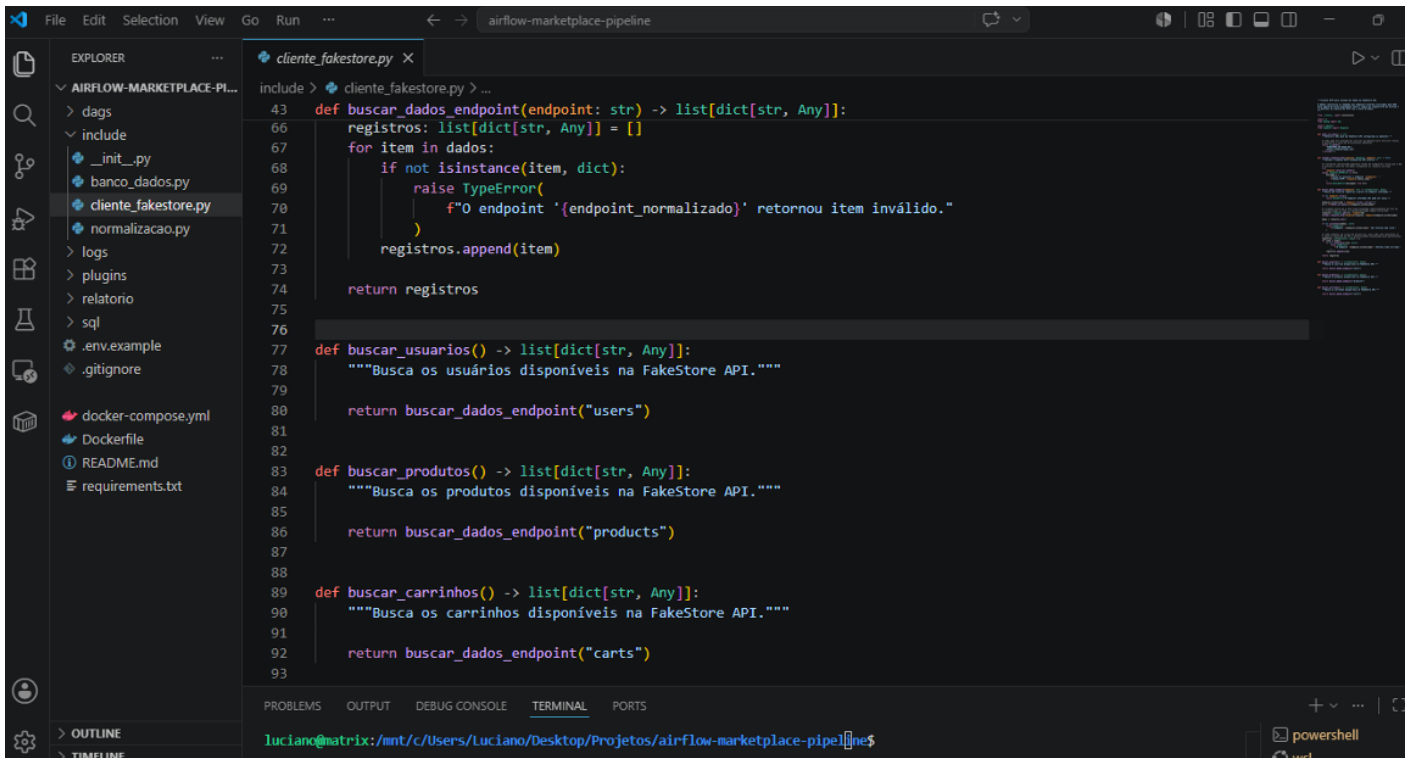
Figura 2 — Script SQL responsável pela criação do schema marketplace e das tabelas operacionais de usuários, produtos e carrinhos.

O script define a estrutura relacional utilizada para armazenar os dados capturados da FakeStore API, separando usuários, produtos e carrinhos em tabelas operacionais do schema marketplace.



```
1 """Cliente HTTP para consumo dos dados da FakeStore API.
2
3 O módulo centraliza as chamadas aos endpoints externos utilizados pela DAG.
4 Essa separação evita acoplamento entre a lógica de orquestração do Airflow e
5 os detalhes de comunicação HTTP com a fonte de dados.
6 """
7
8 from __future__ import annotations
9
10 import os
11 from typing import Any
12
13 import requests
14 from requests import Response
15
16
17 def obter_url_base() -> str:
18     """Retorna a URL base da FakeStore API configurada no ambiente."""
19
20     # A URL pode ser alterada por variável de ambiente para facilitar testes,
21     # manutenção e execução em diferentes ambientes.
22     return os.getenv(
23         "FAKESTORE_API_BASE_URL",
24         "https://fakestoreapi.com",
25     ).rstrip("/")
26
27
28 def validar_resposta_http(resposta: Response, endpoint: str) -> None:
29     """Valida a resposta HTTP recebida da API externa."""
```

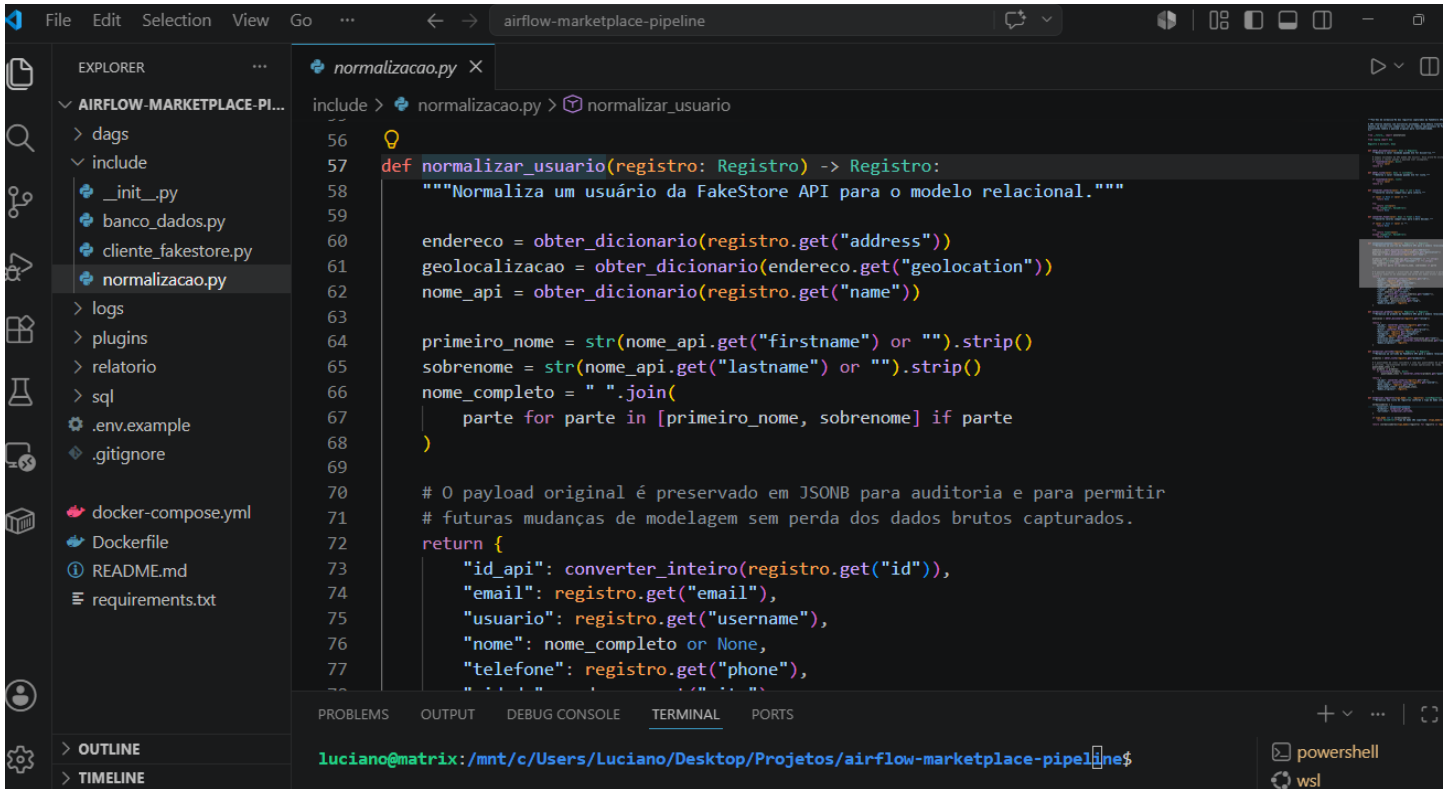
Figura 3 — Módulo cliente_fakestore.py responsável pela configuração da URL base e validação das respostas HTTP da FakeStore API.



```
include > cliente_fakestore.py > ...
43 def buscar_dados_endpoint(endpoint: str) -> list[dict[str, Any]]:
66     registros: list[dict[str, Any]] = []
67     for item in dados:
68         if not isinstance(item, dict):
69             raise TypeError(
70                 f'O endpoint '{endpoint_normalizado}' retornou item inválido.'
71             )
72         registros.append(item)
73     return registros
74
75
76
77 def buscar_usuarios() -> list[dict[str, Any]]:
78     """Busca os usuários disponíveis na FakeStore API."""
79
80     return buscar_dados_endpoint("users")
81
82
83 def buscar_produtos() -> list[dict[str, Any]]:
84     """Busca os produtos disponíveis na FakeStore API."""
85
86     return buscar_dados_endpoint("products")
87
88
89 def buscar_carrinhos() -> list[dict[str, Any]]:
90     """Busca os carrinhos disponíveis na FakeStore API."""
91
92     return buscar_dados_endpoint("carts")
93
```

Figura 4 — Funções tipadas de captura dos dados de usuários, produtos e carrinhos da FakeStore API.

O módulo concentra a integração com os endpoints da FakeStore API e disponibiliza funções específicas para cada tipo de dado utilizado no pipeline.



```
include > normalizacao.py > normalizar_usuario
56
57 def normalizar_usuario(registro: Registro) -> Registro:
58     """Normaliza um usuário da FakeStore API para o modelo relacional."""
59
60     endereco = obter_dicionario(registro.get("address"))
61     geolocalizacao = obter_dicionario(endereco.get("geolocation"))
62     nome_api = obter_dicionario(registro.get("name"))
63
64     primeiro_nome = str(nome_api.get("firstname") or "").strip()
65     sobrenome = str(nome_api.get("lastname") or "").strip()
66     nome_completo = " ".join(
67         parte for parte in [primeiro_nome, sobrenome] if parte
68     )
69
70     # O payload original é preservado em JSONB para auditoria e para permitir
71     # futuras mudanças de modelagem sem perda dos dados brutos capturados.
72     return {
73         "id_api": converter_inteiro(registro.get("id")),
74         "email": registro.get("email"),
75         "usuario": registro.get("username"),
76         "nome": nome_completo or None,
77         "telefone": registro.get("phone"),
78     }
```

Figura 5 — Módulo de normalização dos dados recebidos da FakeStore API, preparando os registros para persistência no modelo relacional.

A normalização transforma estruturas aninhadas da API em registros compatíveis com as tabelas do banco operacional, preservando também o payload original para rastreabilidade.

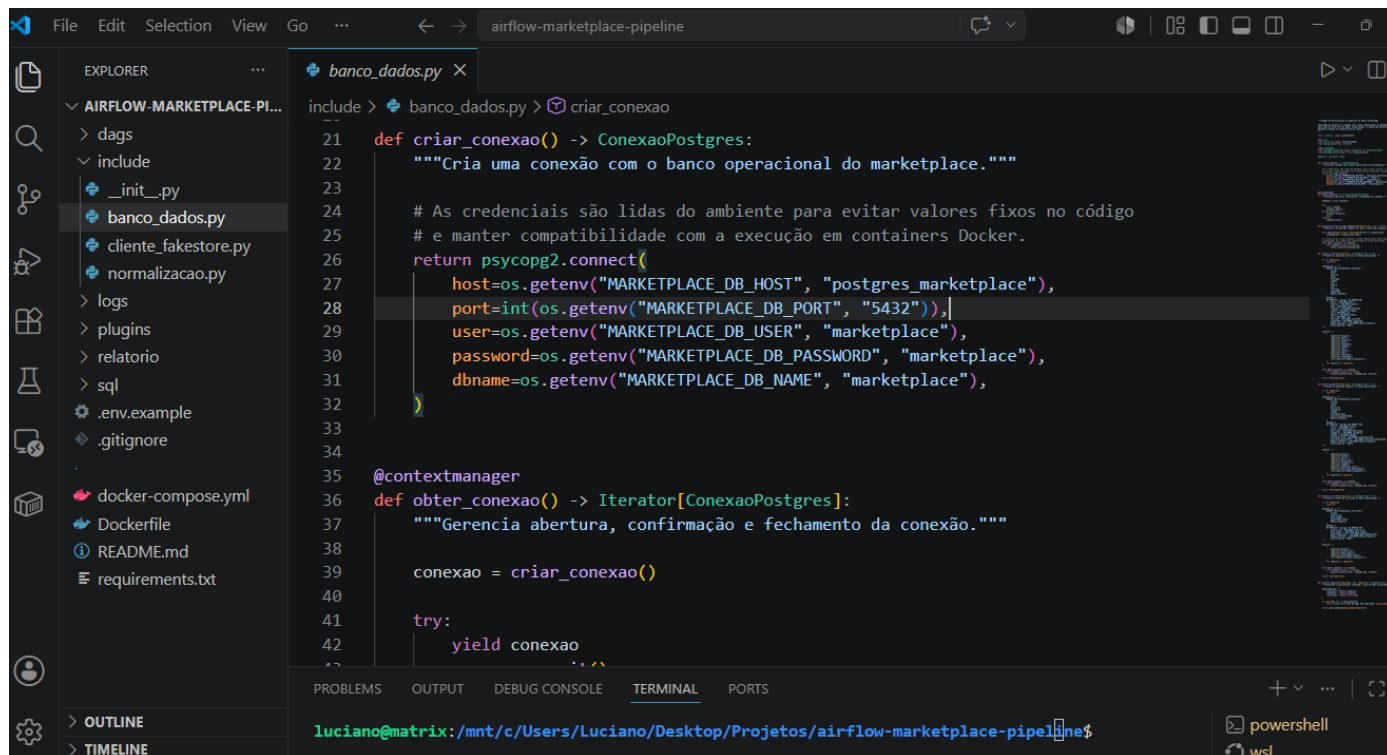


Figura 6 — Camada de persistência responsável pela conexão com o PostgreSQL e gravação dos dados normalizados no banco operacional.

O módulo centraliza a abertura de conexão, execução do script SQL e inserção dos registros normalizados nas tabelas de usuários, produtos e carrinhos.

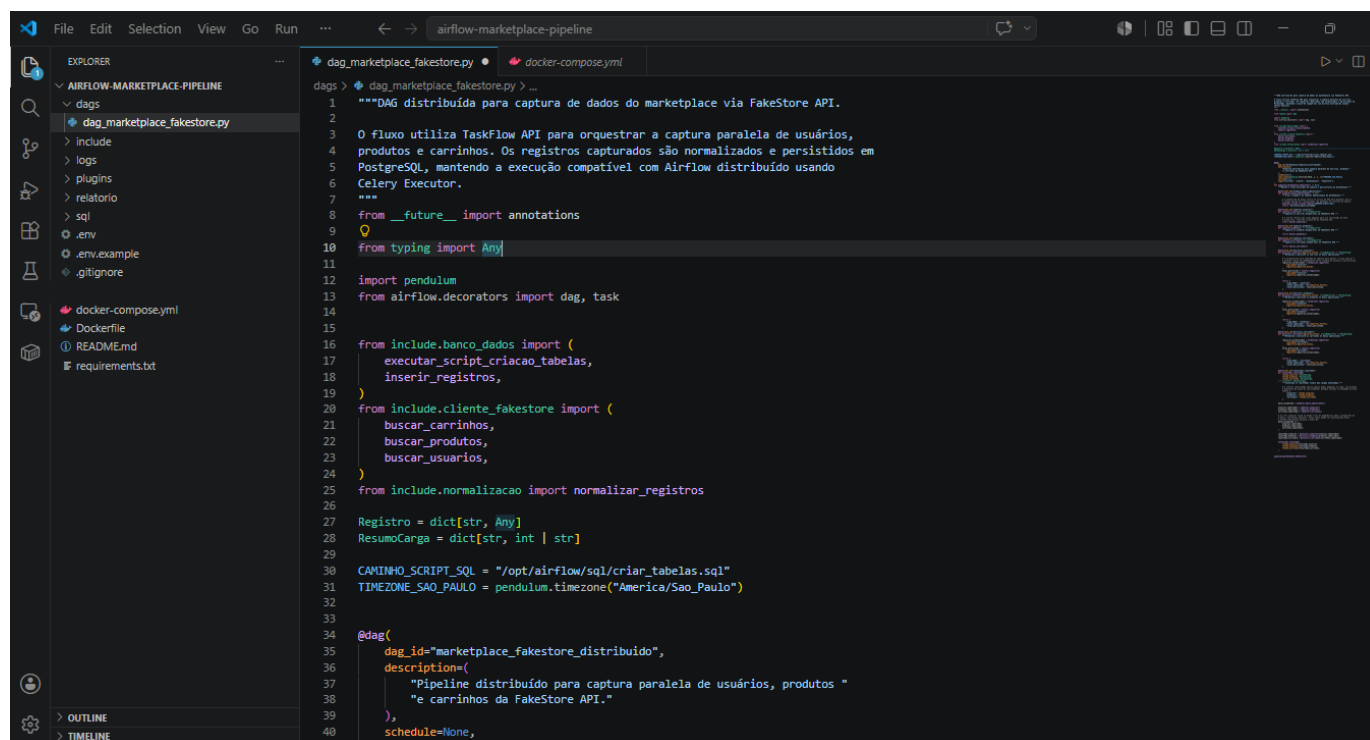
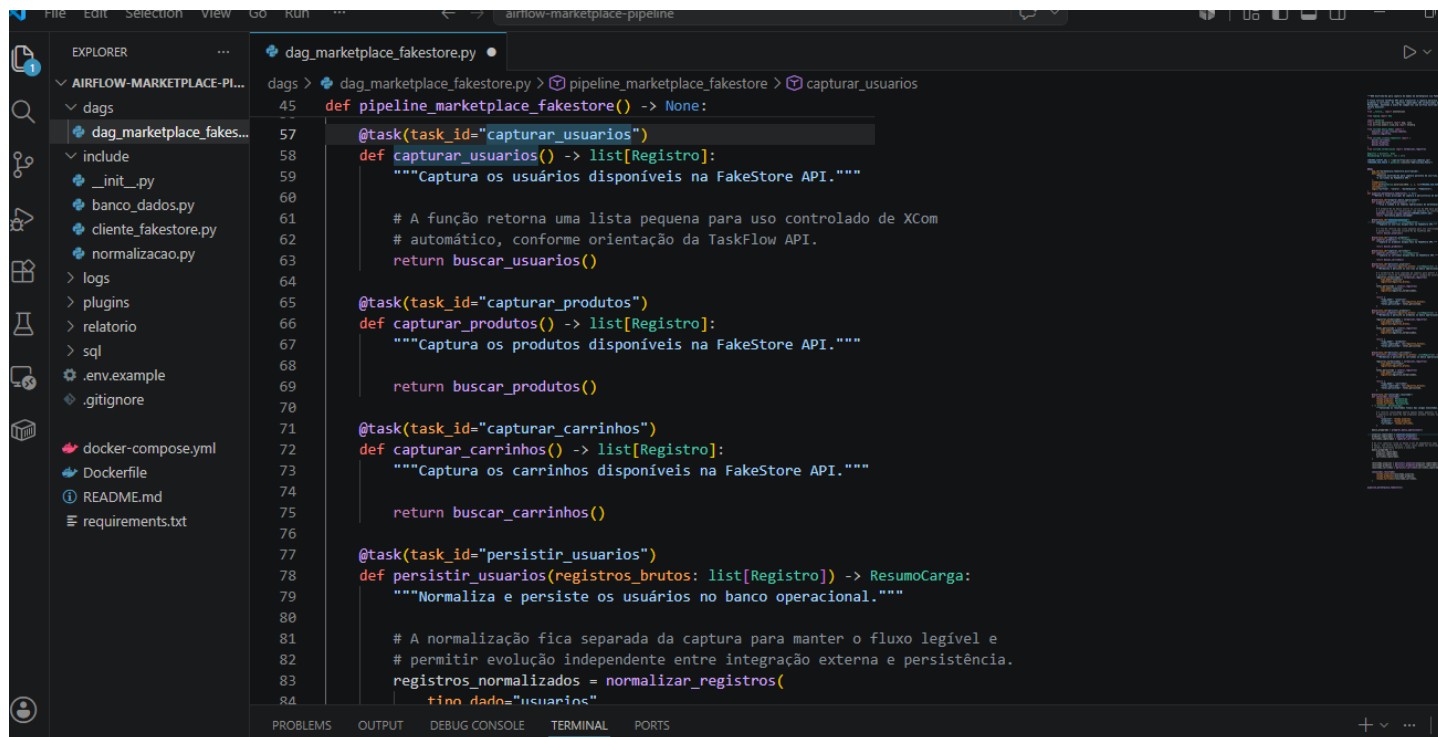


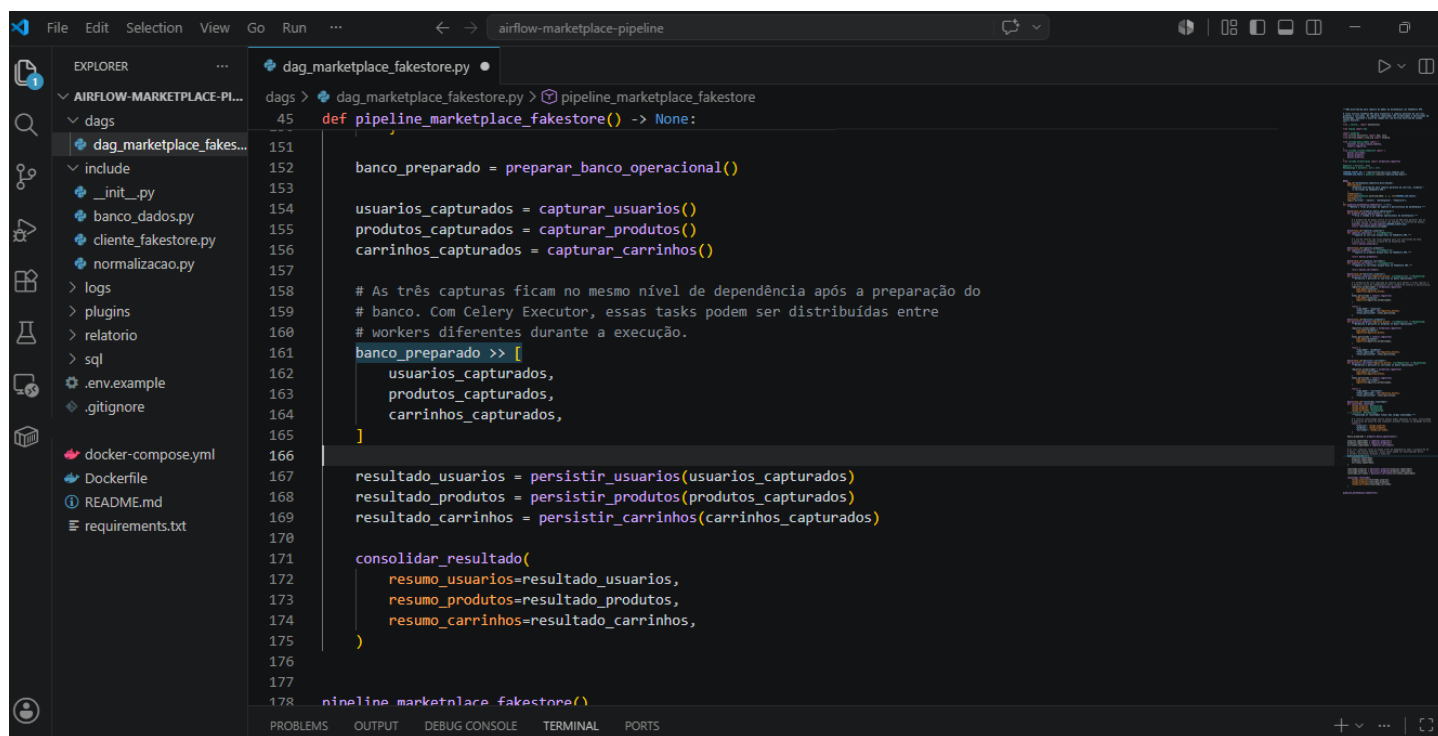
Figura 7 — Definição da DAG principal com TaskFlow API, timezone America/Sao_Paulo, start_date e catchup=False atendendo os requisitos de agendamento.



```
45 def pipeline_marketplace_fakestore() -> None:
46
47     @task(task_id="capturar_usuarios")
48     def capturar_usuarios() -> list[Registro]:
49         """Captura os usuários disponíveis na FakeStore API."""
50
51         # A função retorna uma lista pequena para uso controlado de XCom
52         # automático, conforme orientação da TaskFlow API.
53         return buscar_usuarios()
54
55     @task(task_id="capturar_produtos")
56     def capturar_produtos() -> list[Registro]:
57         """Captura os produtos disponíveis na FakeStore API."""
58
59         return buscar_produtos()
60
61     @task(task_id="capturar_carrinhos")
62     def capturar_carrinhos() -> list[Registro]:
63         """Captura os carrinhos disponíveis na FakeStore API."""
64
65         return buscar_carrinhos()
66
67     @task(task_id="persistir_usuarios")
68     def persistir_usuarios(registros_brutos: list[Registro]) -> ResumoCarga:
69         """Normaliza e persiste os usuários no banco operacional."""
70
71         # A normalização fica separada da captura para manter o fluxo legível e
72         # permitir evolução independente entre integração externa e persistência.
73         registros_normalizados = normalizar_registros(
74             registros_brutos,
75             tipo_dado="usuarios"
76         )
```

Figura 8 — Tasks de captura dos dados de usuários, produtos e carrinhos da FakeStore API.

As tasks foram implementadas com TaskFlow API e retornam dados por meio de XCom automático, mantendo apenas informações necessárias para a continuidade do fluxo.



```
151 banco_preparado = preparar_banco_operacional()
152
153 usuarios_capturados = capturar_usuarios()
154 produtos_capturados = capturar_produtos()
155 carrinhos_capturados = capturar_carrinhos()
156
157 # As três capturas ficam no mesmo nível de dependência após a preparação do
158 # banco. Com Celery Executor, essas tasks podem ser distribuídas entre
159 # workers diferentes durante a execução.
160 banco_preparado >> [
161     usuarios_capturados,
162     produtos_capturados,
163     carrinhos_capturados,
164 ]
165
166 resultado_usuarios = persistir_usuarios(usuarios_capturados)
167 resultado_produtos = persistir_produtos(produtos_capturados)
168 resultado_carrinhos = persistir_carrinhos(carrinhos_capturados)
169
170 consolidar_resultado(
171     resumo_usuarios=resultado_usuarios,
172     resumo_produtos=resultado_produtos,
173     resumo_carrinhos=resultado_carrinhos,
174 )
175
176
177
178 pipeline_marketplace_fakestore()
```

Figura 9 — Definição das dependências da DAG, permitindo a execução paralela das capturas após a preparação do banco operacional.

Após a preparação do banco, as três tarefas de captura são organizadas no mesmo nível de dependência, permitindo execução paralela no ambiente distribuído com Celery Executor.

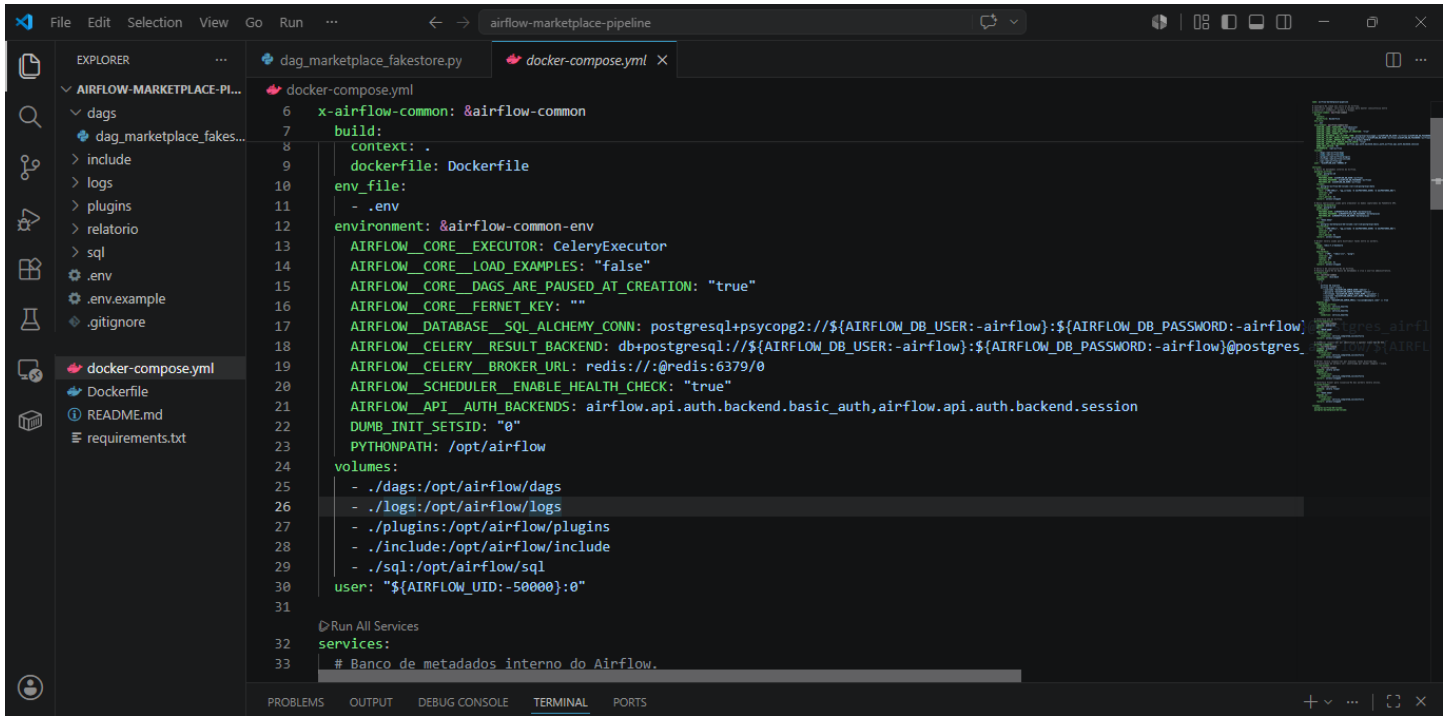


Figura 10 — Configuração dos serviços Airflow com Celery Executor, broker Redis e volumes compartilhados do projeto.

A configuração define o executor distribuído do Airflow, o Redis como broker Celery e o mapeamento dos diretórios locais dags, logs, plugins, include e sql para os containers.

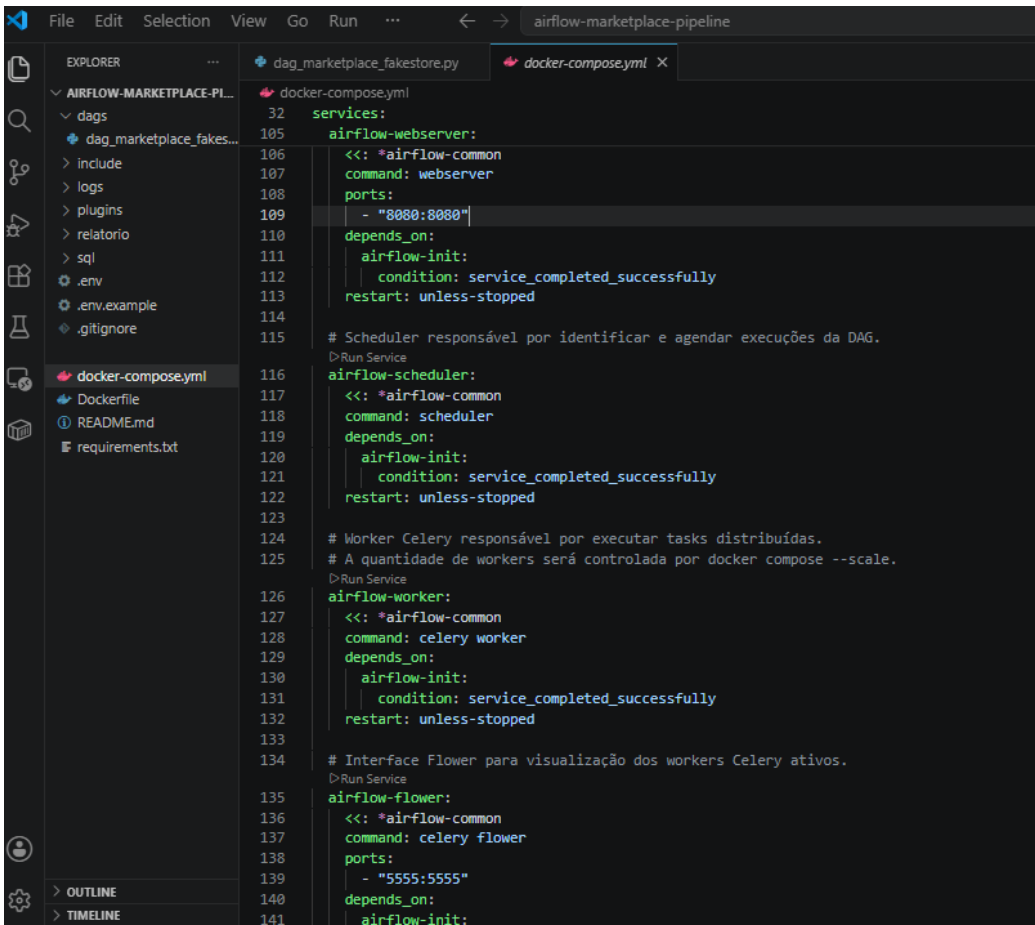


Figura 11 — Serviços da arquitetura distribuída do Airflow, incluindo Webserver, Scheduler, Worker Celery e Flower.

```
airflow-init-1 | [2026-06-28T19:36:12.242+0000] {override.py:1597} INFO - Added user admin
airflow-init-1 | User "admin" created with role "Admin"
```

Figura 12 — Inicialização do banco de metadados do Airflow e criação do usuário administrativo do ambiente.

```
luciano@matrix: /mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$ docker compose up -d --scale airflow-worker=3
[+] up 10/10
✓ Container airflow-marketplace-pipeline-postgres_marketplace-1 Healthy 0.6s
✓ Container airflow-marketplace-pipeline-postgres_airflow-1 Healthy 0.6s
✓ Container airflow-marketplace-pipeline-redis-1 Healthy 0.6s
✓ Container airflow-marketplace-pipeline-airflow-flower-1 Running 0.0s
✓ Container airflow-marketplace-pipeline-airflow-scheduler-1 Running 0.0s
✓ Container airflow-marketplace-pipeline-airflow-webserver-1 Running 0.0s
✓ Container airflow-marketplace-pipeline-airflow-worker-1 Running 0.0s
✓ Container airflow-marketplace-pipeline-airflow-worker-2 Running 0.0s
✓ Container airflow-marketplace-pipeline-airflow-worker-3 Running 0.0s
✓ Container airflow-marketplace-pipeline-airflow-init-1 Exited 14.3s
```

Figura 13 — Inicialização da arquitetura distribuída do Airflow com PostgreSQL, Redis, Webserver, Scheduler, Flower e três workers Celery. Demonstra que três containers de workers Celery foram disponibilizados para execução distribuída das tarefas.

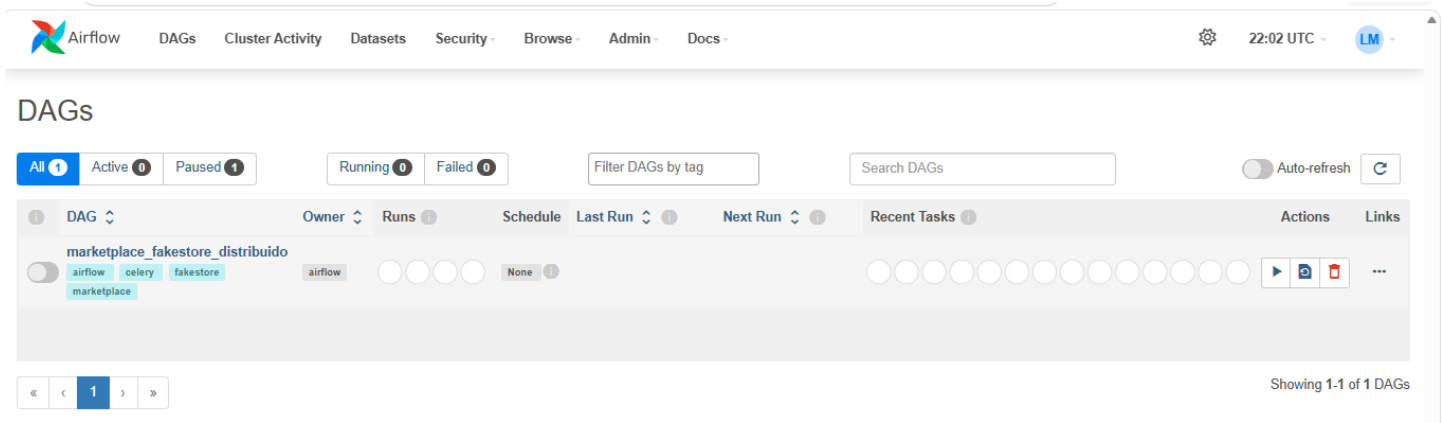


Figura 14 — Interface do Apache Airflow exibindo a DAG marketplace_fakestore_distribuido carregada e pausada antes da execução manual. Como o ambiente foi configurado para criar DAGs inicialmente pausadas, foi necessária ativação manual antes do disparo do pipeline.

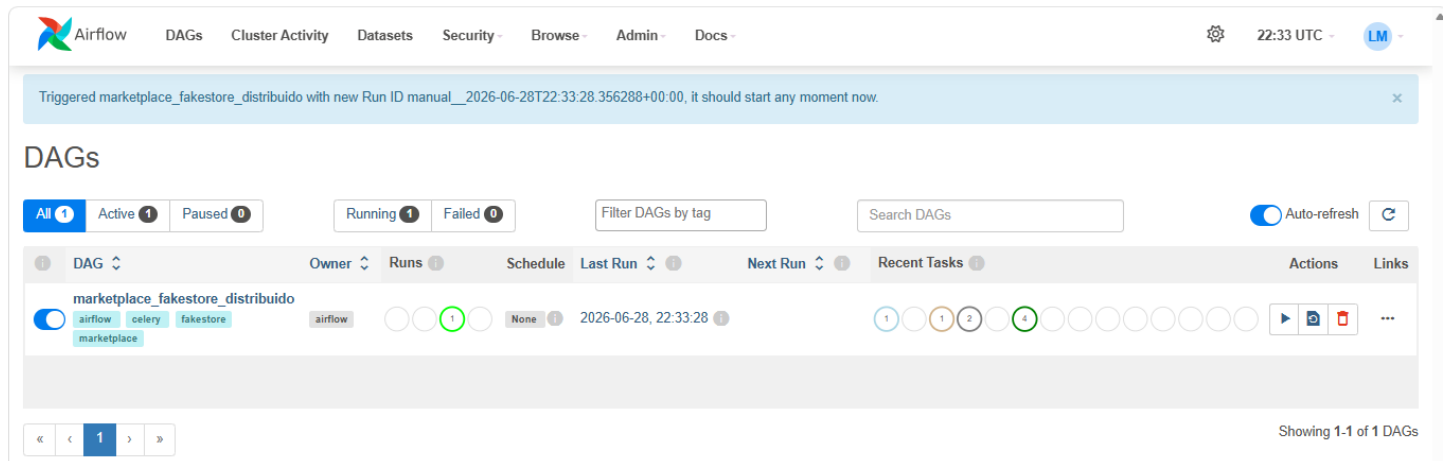


Figura 15 — DAG marketplace_fakestore_distribuido ativada e execução manual iniciada na interface do Apache Airflow.

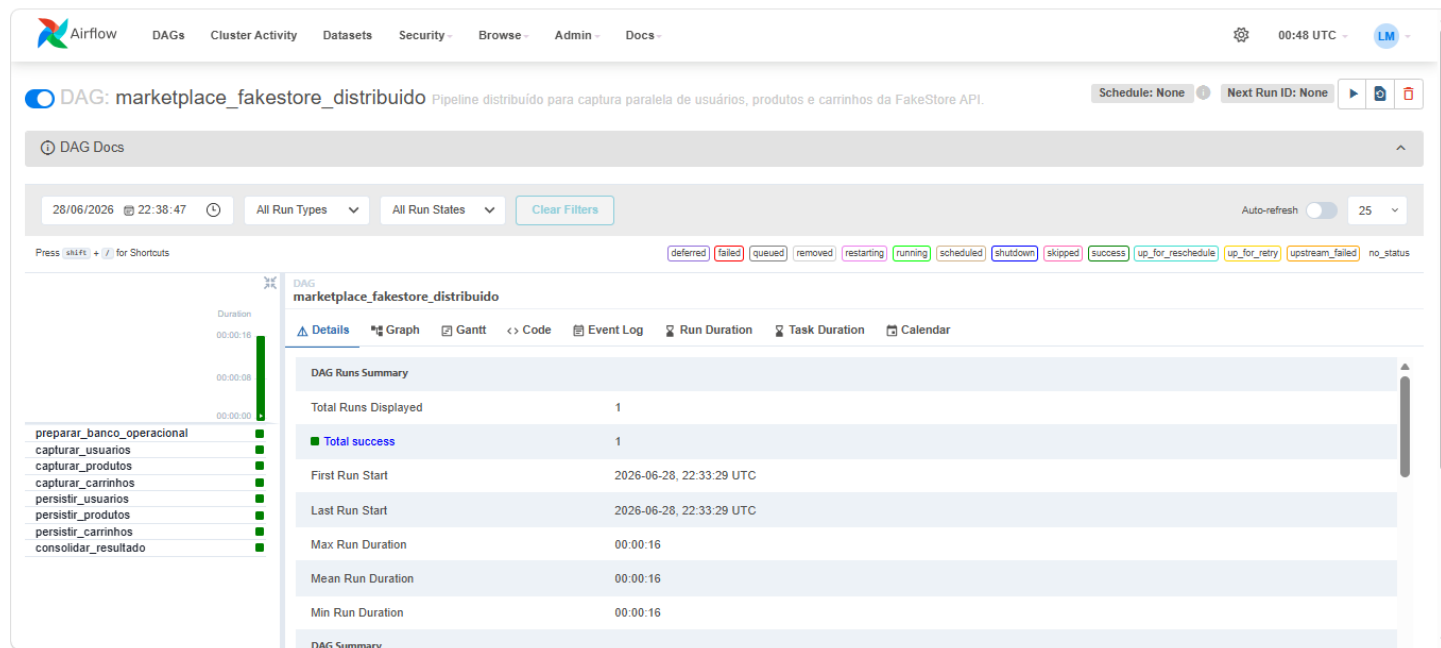


Figura 16 — Resumo da execução da DAG marketplace_fakestore_distribuido concluída com sucesso no Apache Airflow, apresentando na tela Total success 1 e todas as tarefas com status verde, indicando que as etapas de preparação do banco, captura, normalização, persistência e consolidação foram executadas corretamente.

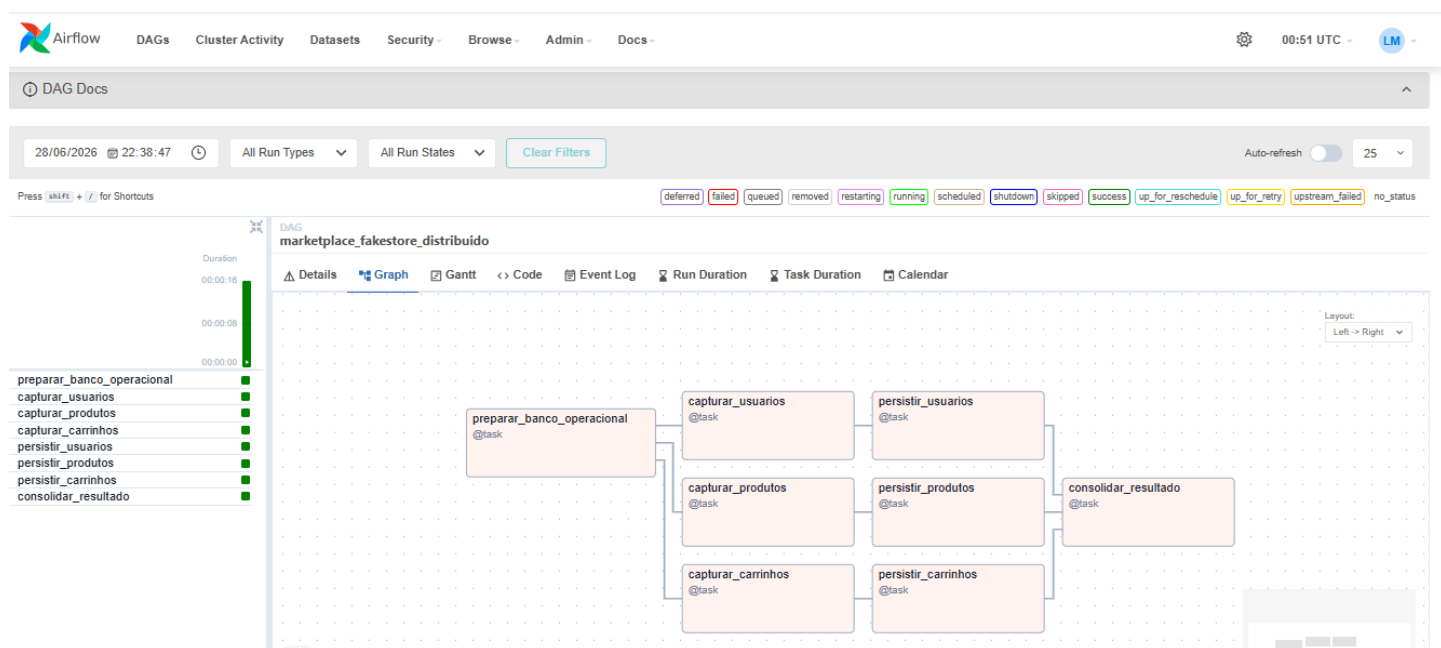


Figura 17 - O Graph apresenta a estrutura lógica do pipeline. Após a preparação do banco operacional, as tarefas de captura de usuários, produtos e carrinhos são executadas em paralelo, seguindo para as respectivas etapas de persistência e consolidação.

The screenshot shows the Flower web interface with a navigation bar (Flower, Workers, Tasks, Broker, Documentation) and a search bar. Below the search bar, there's a table of workers. The table has columns: Worker, Status, Active, Processed, Failed, Succeeded, Retried, and Load Average. Three workers are listed, all with a status of 'Online'. The 'Total' row shows 0 active workers, 8 processed, 0 failed, 8 succeeded, and 0 retried. The load average for each worker is 0.07, 0.18, 0.24. At the bottom, it says 'Showing 1 to 3 of 3 workers' and has 'Previous' and 'Next' buttons.

Worker	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@e764b1178cf	Online	0	3	0	3	0	0.07, 0.18, 0.24
celery@25e914c784c8	Online	0	3	0	3	0	0.07, 0.18, 0.24
celery@4704d195d05b	Online	0	2	0	2	0	0.07, 0.18, 0.24
Total		0	8	0	8	0	

Figura 18 — Interface Flower exibindo **três** workers Celery ativos no ambiente distribuído do Airflow. A interface Flower apresenta três workers Celery online, confirmando a execução distribuída. A tela também mostra tarefas processadas com sucesso e ausência de falhas.

```

luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$ docker compose stop airflow-worker
[+] stop 3/3
✓ Container airflow-marketplace-pipeline-airflow-worker-1 Stopped
✓ Container airflow-marketplace-pipeline-airflow-worker-3 Stopped
✓ Container airflow-marketplace-pipeline-airflow-worker-2 Stopped
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$

```

Figura 19 — Interrupção dos três containers de workers Celery do Airflow.

```

luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$ docker compose rm -f airflow-worker
Going to remove airflow-marketplace-pipeline-airflow-worker-3, airflow-marketplace-pipeline-airflow-worker-2, airflow-marketplace-pipeline-airflow-worker-1
[+] remove 3/3
✓ Container airflow-marketplace-pipeline-airflow-worker-1 Removed
✓ Container airflow-marketplace-pipeline-airflow-worker-3 Removed
✓ Container airflow-marketplace-pipeline-airflow-worker-2 Removed
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$

```

Figura 20 — Remoção dos containers dos três workers Celery do Airflow.

```

luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE
CREATED    STATUS    PORTS
airflow-marketplace-pipeline-airflow-flower-1    airflow-marketplace-pipeline-airflow-flower    "/usr/bin/dumb-init ..."    airflow-flower
6 hours ago    Up 6 hours    0.0.0.0:5555->5555/tcp, [::]:5555->5555/tcp, 8080/tcp
airflow-marketplace-pipeline-airflow-scheduler-1    airflow-marketplace-pipeline-airflow-scheduler    "/usr/bin/dumb-init ..."    airflow-scheduler
6 hours ago    Up 6 hours    8080/tcp
airflow-marketplace-pipeline-airflow-webserver-1    airflow-marketplace-pipeline-airflow-webserver    "/usr/bin/dumb-init ..."    airflow-webserver
6 hours ago    Up 6 hours    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
airflow-marketplace-pipeline-postgres_airflow-1    postgres:16                                "docker-entrypoint.s..."    postgres_airflow
6 hours ago    Up 6 hours (healthy)    5432/tcp
airflow-marketplace-pipeline-postgres_marketplace-1    postgres:16                                "docker-entrypoint.s..."    postgres_marketplace
6 hours ago    Up 6 hours (healthy)    0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
airflow-marketplace-pipeline-redis-1                redis:7.2-bookworm                        "docker-entrypoint.s..."    redis
6 hours ago    Up 6 hours (healthy)    6379/tcp
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$

```

Figura 21 — Validação da remoção dos workers Celery, mantendo os serviços principais do Airflow ativos.

```
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$ docker compose up -d --scale airflow-worker=5
[+] up 12/12
✓ Container airflow-marketplace-pipeline-redis-1           Healthy      4.0s
✓ Container airflow-marketplace-pipeline-postgres_marketplace-1 Healthy      4.0s
✓ Container airflow-marketplace-pipeline-postgres_airflow-1 Healthy      4.0s
✓ Container airflow-marketplace-pipeline-airflow-flower-1   Running      0.0s
✓ Container airflow-marketplace-pipeline-airflow-webserver-1 Running      0.0s
✓ Container airflow-marketplace-pipeline-airflow-scheduler-1 Running      0.0s
✓ Container airflow-marketplace-pipeline-airflow-worker-5   Started      19.2s
✓ Container airflow-marketplace-pipeline-airflow-worker-1   Started      21.2s
✓ Container airflow-marketplace-pipeline-airflow-worker-2   Started      18.3s
✓ Container airflow-marketplace-pipeline-airflow-worker-3   Started      22.3s
✓ Container airflow-marketplace-pipeline-airflow-worker-4   Started      20.2s
✓ Container airflow-marketplace-pipeline-airflow-init-1     Exited       13.3s
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$
```

Figura 22 - Após a remoção dos workers anteriores, a estrutura distribuída foi escalada para cinco workers Celery.

Flower Workers Tasks Broker Documentation

Show 15 workers Search:

Worker	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@9631d2669fea	Online	0	0	0	0	0	2.07, 1.34, 0.83
celery@f4fc73be419e	Online	0	0	0	0	0	2.07, 1.34, 0.83
celery@fe46c04e346d	Online	0	0	0	0	0	2.07, 1.34, 0.83
celery@428db7d4708f	Online	0	0	0	0	0	2.07, 1.34, 0.83
celery@47bf041c2c56	Online	0	0	0	0	0	2.07, 1.34, 0.83
Total		0	0	0	0	0	

Showing 1 to 5 of 5 workers Previous 1 Next

Figura 23 — Interface Flower exibindo cinco workers Celery ativos após o escalonamento da arquitetura distribuída do Airflow.

```
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$ docker compose up -d --scale airflow-worker=2
[+] up 12/12
✓ Container airflow-marketplace-pipeline-postgres_airflow-1 Healthy      45.3s
✓ Container airflow-marketplace-pipeline-postgres_marketplace-1 Healthy      45.3s
✓ Container airflow-marketplace-pipeline-redis-1           Healthy      45.3s
✓ Container airflow-marketplace-pipeline-airflow-flower-1   Running      0.0s
✓ Container airflow-marketplace-pipeline-airflow-webserver-1 Running      0.0s
✓ Container airflow-marketplace-pipeline-airflow-scheduler-1 Running      0.0s
✓ Container airflow-marketplace-pipeline-airflow-worker-1   Running      0.0s
✓ Container airflow-marketplace-pipeline-airflow-worker-2   Running      0.0s
✓ Container airflow-marketplace-pipeline-airflow-worker-3   Removed      44.4s
✓ Container airflow-marketplace-pipeline-airflow-worker-4   Removed      44.7s
✓ Container airflow-marketplace-pipeline-airflow-worker-5   Removed      44.5s
✓ Container airflow-marketplace-pipeline-airflow-init-1     Exited       58.0s
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$
```

Figura 24 — Redução da arquitetura distribuída do Airflow para dois workers Celery sem interromper os serviços principais.

Flower Workers Tasks Broker Documentation

Show 15 workers Search:

Worker	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@fe46c04e346d	Online	0	0	0	0	0	0.31, 0.37, 0.55
celery@428db7d4708f	Online	0	0	0	0	0	0.31, 0.37, 0.55
celery@9631d2669fea	Offline	0	0	0	0	0	N/A
celery@f4fc73be419e	Offline	0	0	0	0	0	N/A
celery@47bf041c2c56	Offline	0	0	0	0	0	N/A
Total		0	0	0	0	0	

Showing 1 to 5 of 5 workers Previous 1 Next

Figura 25 — Interface Flower após a redução da arquitetura para dois workers Celery ativos.

```
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline$ docker compose ps airflow-worker
```

NAME	IMAGE	COMMAND	SERVICE	CREATED
airflow-marketplace-pipeline-airflow-worker-1	airflow-marketplace-pipeline-airflow-worker	"/usr/bin/dumb-init ..."	airflow-worker	17 hours ago
Up 17 hours	8080/tcp			
airflow-marketplace-pipeline-airflow-worker-2	airflow-marketplace-pipeline-airflow-worker	"/usr/bin/dumb-init ..."	airflow-worker	17 hours ago
Up 17 hours	8080/tcp			

Figura 26 — Validação no Docker Compose confirmando apenas dois workers Celery em execução.

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
<pre>"SELECT 'usuarios' AS tabela, COUNT(*) AS total FROM marketplace.usuarios UNION ALL SELECT 'produtos' AS tabela, COUNT(*) AS total FROM marketplace.produtos UNION ALL SELECT 'carrinhos' AS tabela, COUNT(*) AS total FROM marketplace.carrinhos;" tabela total ----- ----- usuarios 10 produtos 20 carrinhos 7 (3 rows)</pre>				
luciano@matrix:/mnt/c/Users/Luciano/Desktop/Projetos/airflow-marketplace-pipeline\$				

Figura 27 — Validação da persistência dos dados capturados no banco PostgreSQL operacional.

A consulta ao banco PostgreSQL operacional confirma a persistência dos dados capturados da FakeStore API. Foram gravados 10 registros de usuários, 20 registros de produtos e 7 registros de carrinhos, acertando que o pipeline executou captura, normalização e carga dos dados nas tabelas do schema marketplace